JAVA CORE CONCEPTS

Basic Input and Output in Java

Console stream formatting, print mechanics, and user input handling via Scanner

> In any programming language, understanding how to **display output** and **take input** forms the bedrock of interaction between your application and the user. In Java, console I/O is managed primarily through stream objects and utility classes like `System.out` and `Scanner`.

1. Java Output Stream Methods

Java sends data to standard output (the console stream) using three core methods from `System.out`:

- `System.out.print()` — Outputs text while retaining cursor position on the same line.
- `System.out.println()` — Outputs text and appends a line break, moving the cursor to the next line.
- `System.out.printf()` — Outputs formatted text using C-style specifiers and placeholders.

Component Anatomy:

- `System` → A built-in class in `java.lang`.
- `out` → A static print stream object connected to the standard output.
- `print / println / printf` → Invoked methods used to stream data to standard output.

Example 1: Basic Console Output

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Welcome to CodeHelp ONE!");
    }
}
```

```
Output:
Welcome to CodeHelp ONE!
```

Explanation: The text inside double quotes represents a `String` literal. `println()` prints the text and moves the cursor to the subsequent line.

2. Comparing print(), println(), and printf()

Example 2: Difference Between print() and println()

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Step 1: Learn");
        System.out.println("Step 2: Practice");
        System.out.print("Phase A ");
        System.out.print("Phase B");
    }
}
```

```
Output:
Step 1: Learn
Step 2: Practice
Phase A Phase B
```

Explanation: `println()` executes line breaks after each call. `print()` maintains the cursor on the current line, causing `Phase A` and `Phase B` to display side by side on the same line.

Example 3: Formatted Output with printf()

`printf()` formats output using placeholder format specifiers, ideal for reports, alignment, and precision formatting.

```java
public class Main {
    public static void main(String[] args) {
        String studentName = "Aman";
        int solvedProblems = 150;
        System.out.printf("Student: %s, Problems Solved: %d", studentName, solvedProblems);
    }
}
```

```
Output:
Student: Aman, Problems Solved: 150
```

Format Placeholders: `%s` evaluates to String data; `%d` evaluates to integer values; `%f` evaluates to floating-point numbers.

3. Printing Variables & String Concatenation

Printing Literals vs. Variables

```java
public class Main {
    public static void main(String[] args) {
        int activeUsers = 2500;
        double averageRating = 4.8;

        System.out.println(activeUsers);    // Correct: prints value 2500
        System.out.println("activeUsers");  // Incorrect: prints text "activeUsers"
    }
}
```

String Concatenation with + Operator

```java
public class Main {
    public static void main(String[] args) {
        String userName = "Riya";
        int contestCount = 12;

        System.out.println("Hello " + userName);
        System.out.println("Contests attempted: " + contestCount);
    }
}
```

Mechanics: When operating alongside a String, the `+` operator switches from addition to string concatenation, converting primitive values into text strings automatically.

4. Taking Input via Scanner Class

Java uses `java.util.Scanner` to parse user input from the keyboard stream (`System.in`).

Step 1: Import Scanner Class

```
import java.util.Scanner;
```

Step 2: Instantiate Scanner Object

```
Scanner sc = new Scanner(System.in);
```

Example 6: Integer Console Input

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total problems solved: ");
        int problems = sc.nextInt();
        System.out.println("You solved " + problems + " problems.");

        sc.close(); // Prevents memory resource leaks
    }
}
```

5. Scanner Data Reading Methods

Method Name	Data Type Parsed	Input Description
`nextInt()`	`int`	Reads standard integer values
`nextLong()`	`long`	Reads large integer values
`nextFloat()`	`float`	Reads single-precision decimal values
`nextDouble()`	`double`	Reads double-precision decimal values
`next()`	`String`	Reads a single token (word up to whitespace)
`nextLine()`	`String`	Reads the entire input line (including spaces)

Example 7: Multiple Input Types Demonstration

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter accuracy percentage: ");
        float accuracy = sc.nextFloat();

        System.out.print("Enter platform name: ");
        String platform = sc.next();

        System.out.println("Accuracy: " + accuracy);
        System.out.println("Platform: " + platform);

        sc.close();
    }
}
```

⚠️ **Critical Difference: next() vs nextLine()**

- `next()` stops reading as soon as it encounters any whitespace (space, tab, newline).
- `nextLine()` reads all characters until the line break is reached.

Mixing primitive readers (like `nextInt()`) before `nextLine()` can consume trailing newline characters prematurely. This is a frequently tested interview trap.

💡 **Placement & Coding Round Importance**

In online assessment platforms (e.g., HackerRank, LeetCode, CodeChef):

- **Input Parsing Efficiency:** Assessment testcases feed inputs sequentially via `System.in`.
- **Parsing Errors:** Mismatched scanner calls cause `InputMismatchException` or runtime crashes.
- **Formatting Requirements:** Strictly matching required output whitespace or line breaks using `printf` prevents failed testcases.

📝 **Practice Exercise**

Build a Java program that:

1. Takes the user's name (`String`)
2. Takes the total problems solved (`int`)
3. Calculates a recommended weekly target (e.g., `problems * 1.25`)
4. Prints a formatted summary report using `printf()`

Run your code in a Java compiler to observe input stream parsing and output layout.